

Aurora Higher Education and Research Academy

School of Engineering

B. Tech – Sophomore Year – CSE

Term – II

JVM Monitoring and Profiling Framework

Advanced Java Profiler and Debugger

Course Name: Java Programming

Faculty: Dr. Vemula Aruna
Associate Professor

Course Code: CSE

LEARNING GROUP: 04

Student Names	Reg. No.
M N Risheendra	241U1R1080
Kattela Jwalin	241U1R1088
Ankit Lama	241U1R1089

Presentation Outline

- 1 Abstract
- 2 Introduction
- 3 Literature Survey
- 4 Project Design
- 5 Methodology
- 6 Implementation
- 7 Results
- 8 Discussion
- 9 Conclusion
- 10 References

Abstract

Modern Java applications are increasingly complex, making it hard to detect **performance bottlenecks**, **memory leaks**, and **runtime errors** manually.

This project presents an **Advanced Java Profiler & Debugger** — a tool that monitors Java applications in real time and delivers structured insights through a graphical interface.

- **Runtime Analysis:** tracks execution time and call frequency.
- **Call Graph:** shows relationships between methods.
- **Memory Analysis:** finds heap-heavy objects and leaks.
- **JShell Debugger:** test code without restarting apps.

Uses **Java Instrumentation API** and **JVMTI** to collect JVM data, displayed via **Java Swing** panels.

Keywords

Java Profiling · Runtime Analysis · JVMTI · Call Graph · Memory Analysis · Instrumentation · Performance Monitoring · Debugging

Problem Statement

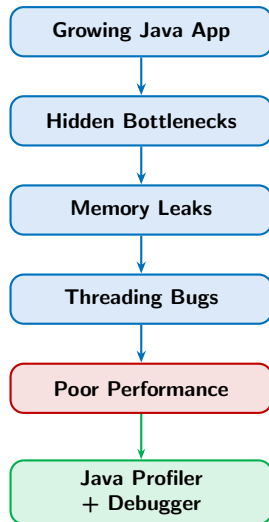
As Java programs scale, analyzing execution manually becomes nearly impossible. Developers face immense challenges in identifying performance issues, detecting logical errors, and understanding execution flow.

The core issues include:

- **Manual Complexity:** Large programs are extremely difficult to analyze by hand.
- **Hidden Bottlenecks:** Performance issues and memory leaks are hard to detect without specialized tools.
- **Debugging Time:** Tracing logical and runtime errors in complex applications is highly time-consuming.
- **Inaccessible Tooling:** Existing enterprise tools are often too complicated for beginners to grasp internally.

Why do we need a Java Profiler?

- Java powers millions of **enterprise apps and web systems**.
- As code grows, **performance issues hide** inside the JVM.
- **Memory leaks** silently degrade application stability.
- **Slow methods** go unnoticed without precise measurement.
- Debugging **multi-threaded** code is notoriously error-prone.
- Existing tools are **complex for beginners** to use and understand.



Introduction: Objectives

The proposed system aims to:

- ① **Understand** the concept of program profiling and debugging in Java.
- ② **Analyze** the execution behaviour of Java applications at runtime.
- ③ **Measure** performance metrics: execution time, call count, memory usage.
- ④ **Identify** errors and inefficiencies in Java programs.
- ⑤ **Develop** a simplified Java profiling and debugging system.
- ⑥ **Visualize** method call graphs, memory maps, and execution snapshots.
- ⑦ **Provide** an educational tool for understanding program execution.
- ⑧ **Enhance** knowledge of software performance monitoring.

Core Philosophy

“Bridging JVM internals with developer-friendly performance insights.”

Introduction: Key Concepts (Part I)

JVM — Java Virtual Machine

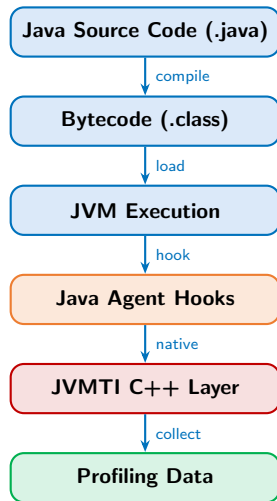
The engine that **executes Java bytecode**. It manages memory, threads, and garbage collection. All profiling happens inside the JVM.

Java Agent

A plugin that **attaches before the application starts**. It intercepts class loading and modifies bytecode without changing source code.

JVMTI — JVM Tool Interface

A **native C++ API** inside the JVM that provides low-level access to threads, memory, method events, and GC callbacks.



Introduction: Key Concepts (Part II)

Bytecode Instrumentation

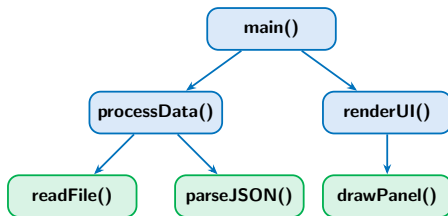
Injecting **monitoring code into bytecode** at class-load time. The application runs normally while extra code records execution details.

Call Graph

A **directed graph** where nodes represent methods and edges show calling relationships. It helps identify execution flow and bottlenecks.

Memory Analysis

Tracks **heap allocation per class/object** using JVMTI callbacks. Helps identify high memory usage and detect leaks.



Call Graph Example

Burchell & Marr — 2023

Don't Trust Your Profiler: Precision & Accuracy of Java Profilers

- Evaluates multiple Java profilers — sampling vs instrumentation.
- **Key Finding:** Different tools produce **inconsistent** results for the same program.
- Instrumentation profilers add overhead; sampling misses short calls.
- **Gap:** No single tool gives reliably accurate data.
- Recommends combining both techniques for better accuracy.

[\[Access Paper\]](#)

Burchell & Marr — 2024

Realistic Results for Instrumentation Profilers in JIT Systems

- Investigates how bytecode instrumentation interferes with **JIT** compilation.
- Extra monitoring code disrupts JIT optimizations.
- **Gap:** Results don't reflect real-world application performance.
- Proposes **hybrid adaptive** profiling methods.
- Selective instrumentation only on critical sections reduces distortion.

[\[Access Paper\]](#)

ASE Research Team — 2024

Comparing CPU & Memory Profiler Results Across Java Versions

- Analyzes how profiling data **varies across different JVM versions**.
- JVM optimizations alter execution behaviour unpredictably.
- **Gap:** Profilers break when new JVM versions are released.
- Cross-version performance comparisons become unreliable.
- Recommends standardized, version-aware profiling frameworks.

[\[Access Paper\]](#)

Programming Journal — 2023

Profiling and Optimizing Java Streams

- Profiles Java **Stream API** operations to detect overhead.
- Parallel streams cause synchronization overhead in many cases.
- Complex pipelines reduce both readability and performance.
- **Gap:** Performance gains depend heavily on dataset size.
- Profiling reveals which stream operations to simplify or replace.

[\[Access Paper\]](#)

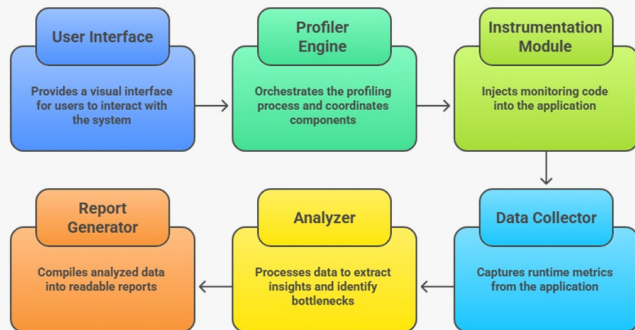
Contemporary Research Innovations (Part I)

Research Title	Authors	Year	Innovation & Impact
Don't Trust Your Profiler: An Empirical Study	T. Burchell, S. Marr	2023	Evaluates profiler accuracy, comparing sampling vs. instrumentation. Shows that profiler outputs need careful interpretation. [View PDF]
Towards Realistic Results for Instrumentation Profilers	T. Burchell, S. Marr	2024	Investigates how instrumentation disrupts JIT compilers. Suggests hybrid and adaptive methods for accurate environments. [View PDF]
Comparing CPU and Memory Profiler Results	Automated SW Eng Team	2024	Analyzes consistency across JVM versions. Proves JVM differences highly impact profiling data reliability. [View Link]

Contemporary Research Innovations (Part II)

Research Title	Authors	Year	Innovation & Impact
Profiling and Optimizing Java Streams	Programming Journal	2023	Profiles Java Stream operations. Identifies inefficient stream operations to optimize performance while maintaining readability. [View Link]
JVM Optimization: Empirical Analysis	SoftwareX Research	2024	Evaluates how JVM configurations (heap size, GC) affect web application performance and resource efficiency. [View Link]
JVM Profiler & Debugger	LG 04	2026	Integrates JVMTI + Java Agent + Swing UI for comprehensive real-time profiling with call graph and memory analysis.

Profiling System Architecture



The architecture has **three layers**:

Frontend

ProfilerUI — displays call graphs, memory maps, and execution snapshots.

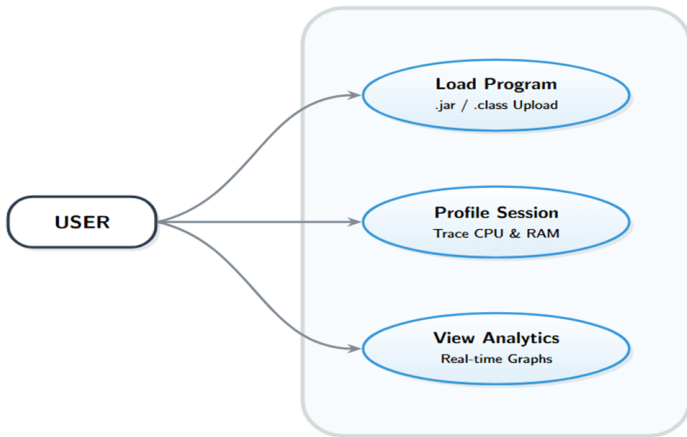
Processing

Profiler Engine coordinates operations; DataSender transmits data.

JVM Layer

Java Agent + JVMTI C++ agent hook into JVM to collect raw data.

JAVA PROFILER SYSTEM



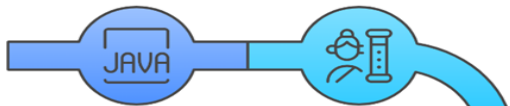
Actor: User / Developer

- **Load Program** — upload target Java application.
- **Profile Session** — start monitoring CPU, memory, method calls.
- **View Call Graph** — visualize method relationships.
- **Analyze Memory** — inspect heap usage per class.
- **Use JShell** — run interactive code snippets.
- **View Analytics** — graphs, timelines, and snapshots.

Project Design: System Workflow

Application Launch

Target Java application
launched with profiling
agent



Agent Initialization

ProfilerAgent initializes
and registers hooks

Data Collection

Profiling data stored in
ProfilerRegistry and
MethodStats



Method Interception

MethodInterceptor
captures runtime
execution

Native Layer Interaction

JVMTI-based C++ agent
interacts with JVM
internals

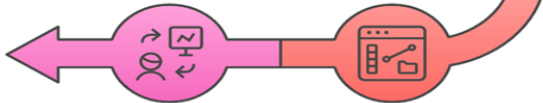


Data Transmission

DataSender sends
profiling data to UI module

User Interaction

Users analyze execution
and view performance
insights



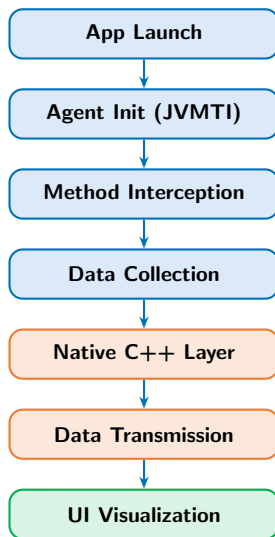
Data Visualization

UI displays call graph,
memory usage, and
snapshots

Methodology: Overview

The system follows a **structured 8-step pipeline** from program execution to real-time visualization.

- ➊ **Application Launch** — app starts with agent attached via `-javaagent`.
- ➋ **Agent Initialization** — JVMTI hooks for method entry/exit enabled.
- ➌ **Method Interception** — every method call captured with timestamp.
- ➍ **Data Collection** — stored in `ProfilerRegistry` and `MethodStats`.
- ➎ **Native Layer** — C++ JVMTI agent collects memory and thread data.
- ➏ **Data Transmission** — `DataSender` sends data via socket.
- ➐ **Visualization** — `ProfilerUI` renders call graph, memory map, snapshots.
- ➑ **User Interaction** — load, analyze, and explore results.

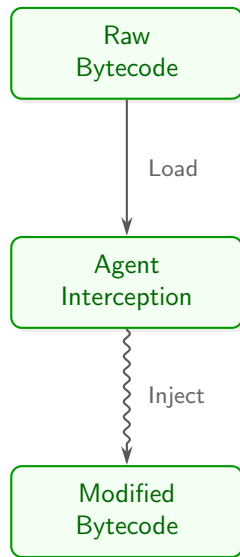


Bytecode Modification Theory

- At load time, the `ClassFileTransformer` intercepts raw bytes.
- Logical *Hooks* are injected at entry/exit points of designated methods.

Data Aggregation Logic

- Continuous data stream is processed using concurrent hash-based structures to ensure thread safety without halting the JVM.

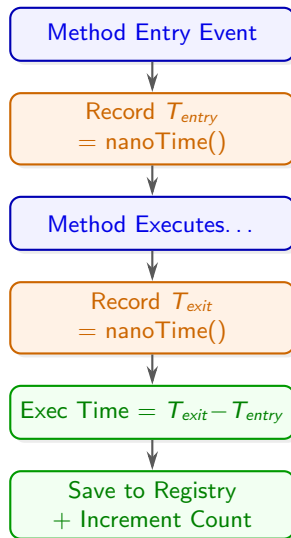


1. Method Execution Tracking

- Hook fires on **method entry** and **method exit**.
- Capture `System.nanoTime()` at both points.
- **Execution Time** = $T_{exit} - T_{entry}$
- **Call Count** incremented on every entry event.
- Data stored per method in `MethodStats` object.

2. Call Graph Construction

- Maintains a **thread-local call stack**.
- On entry: push current method; record caller → callee edge.
- On exit: pop method from stack.
- Builds a **directed graph**: Nodes = Methods, Edges = Calls.
- Graph rendered visually in `CallGraphPanel`.



3. Aggregation Algorithm

- Uses **ConcurrentHashMap** for thread-safe storage.
- Key: method signature; Value: MethodStats object.
- Stores **cumulative** total time and total call count.
- Dynamically updates on every intercepted call.
- Enables real-time statistics without locking the app.

4. Memory Tracking (JVMTI)

- JVMTI registers **VMOBJECTAlloc** callback.
- Fires every time any object is allocated on heap.
- Tracks **memory usage per class/module**.
- Data shown in MemoryMapPanel as live chart.

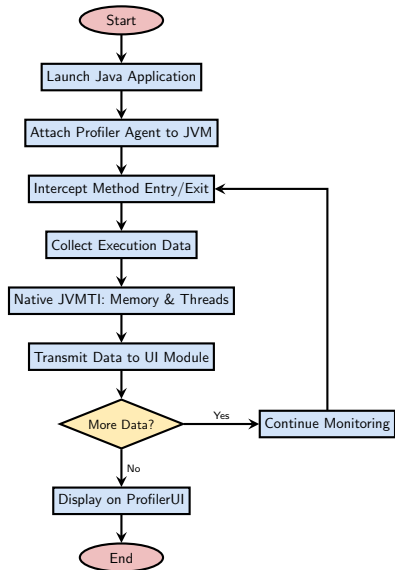
5. Thread Monitoring

- JVMTI captures **ThreadStart** and **ThreadEnd** events.
- Associates each method call with its **Thread ID**.
- Tracks number of active threads and their execution patterns.
- Helps detect **thread contention** and deadlocks.

Performance Metrics Collected

Exec Time	$T_{exit} - T_{entry}$
Call Count	Counter per method
Heap Usage	Per class (bytes)
Call Depth	Graph traversal depth
Thread Count	Active threads at runtime
CPU (indirect)	Freq $\times$ Duration

Methodology: System Flowchart



Core Implementation: Initialization & Interception

Step 1: Agent Initialization (ProfilerAgent.java)

The agent uses the `premain()` method to attach to the JVM before the main app starts. It registers a `ClassFileTransformer` to enable runtime bytecode modification seamlessly without altering the original source code.

Step 2: Method Interception (MethodInterceptor.java)

Target methods are dynamically wrapped to capture precise execution metrics. The flow follows:

- **Pre-execution:** Captures the start time just before execution begins.
- **Execution:** Invokes the original method logic.
- **Post-execution:** Captures the end time safely within a `finally` block.
- **Dispatch:** Calculates the duration and dispatches it to the central registry.

Step 3: Data Collection

`ProfilerRegistry.java`

All intercepted data flows here. It utilizes a `ConcurrentHashMap<String, MethodStats>` to ensure that data inserted concurrently by multiple application threads does not cause race conditions.

Step 4: Statistics Calculation

`MethodStats.java`

Each individual method profile is tracked as an object. It actively increments `callCount` and compounds `totalTime`, forming the mathematical basis for the UI.

Step 5: Data Transmission

`DataSender.java`

Since the monitored application and the visualizer may run independently, custom socket communication (via `localhost:5000`) is established.

This module serializes and transmits the `ProfilerRegistry` statistics to the UI listener continuously in real-time, ensuring the dashboard is always up to date.

Step 6: JVMTI Native Integration (`agent.cpp`)

A native C++ agent is loaded alongside the Java agent using the `Agent_OnLoad` hook. This bypasses standard Java limits, extracting raw garbage collection cycles, heap snapshots, and deep thread states directly from the JVM.

Step 7: UI Initialization

`Launcher.java`

Acts as the entry point for the visualizer. It boots up the Java Swing framework, preparing the canvas and active listeners for incoming socket data.

Step 8: Displaying Profiling Data

`ProfilerUI.java`

Features an `updateData()` routine that unpacks the serialized maps and triggers repaints on:

- **Call Graphs**
- **Execution Time Tables**
- **Memory Usage Bar Charts**

Implementation: Technologies Used (Part I)

Programming Languages

- **Java** — Core agent, UI, registry, data transfer.
- **C++** — JVMTI native agent (`agent.cpp`).

Build Tools

- **Maven** — Java build and dependency management.
- **CMake** — compiling the native C++ JVMTI agent.

Frameworks & APIs

- **JVMTI** — Java Virtual Machine Tool Interface (native).
- **Java Instrumentation API** — method interception.
- **Java Swing** — graphical UI components.

Implementation: Technologies Used (Part II)

UI Components (Java Swing)

- **CallGraphPanel** — visualizes method call graph.
- **MemoryMapPanel** — shows live heap usage per class.
- **SnapshotWindow** — captures execution state at a moment.

Communication

- **Socket-based** agent-to-UI communication.
- Structured ProfilerData objects serialized over socket.
- **Real-time streaming** of profiling data to dashboard.

Dev Environment

Any Java IDE (IntelliJ / Eclipse) + JVM-based execution environment.

Results: System Output Overview

The JVM Profiling Framework was **successfully implemented and tested** across multiple Java programs of varying complexity.

What Was Verified

- Profiler correctly captures **method execution details**.
- Call graph accurately shows **method relationships**.
- Memory panel identifies **heap-heavy objects**.
- JShell debugger enables **interactive testing**.

3 Key Output Screens

- ① **Method Execution + Call Graph** panel.
- ② **JShell Debugger** interactive output.
- ③ **Memory Usage Analysis** heap chart.



Results: Method Execution and Call Graph

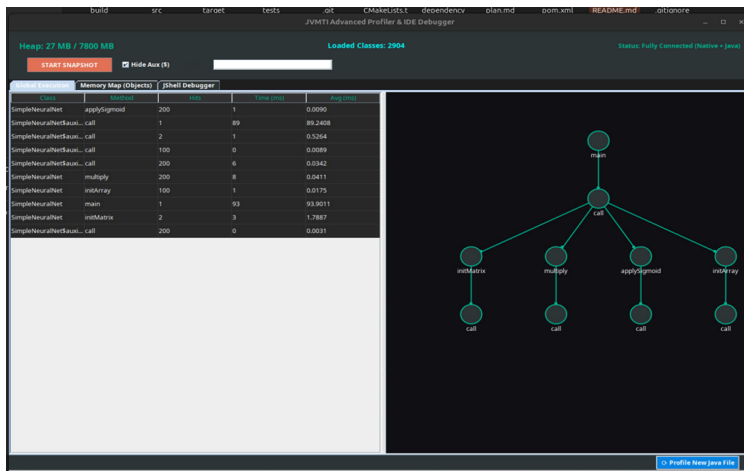


Figure: Displays **execution time**, **call count**, and the **call graph**. **Insight:** Highlights performance bottlenecks.

Results: JShell Debugger Output

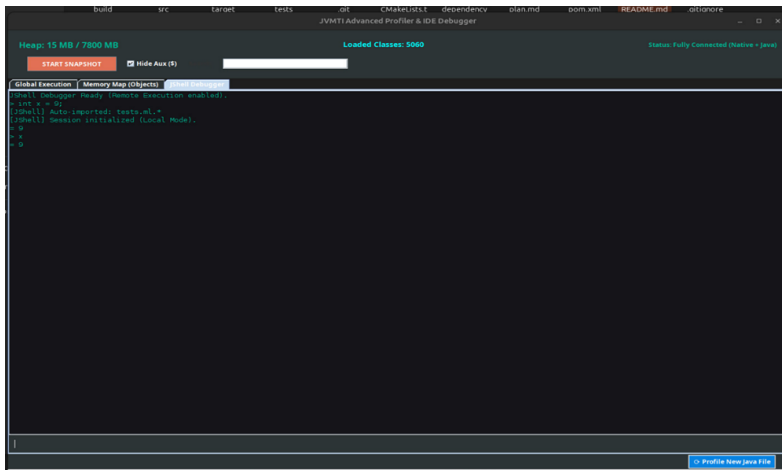


Figure: Interactive **JShell execution** for testing code snippets instantly. **Benefit:** Faster debugging and quick validation.

Results: Memory Usage Analysis

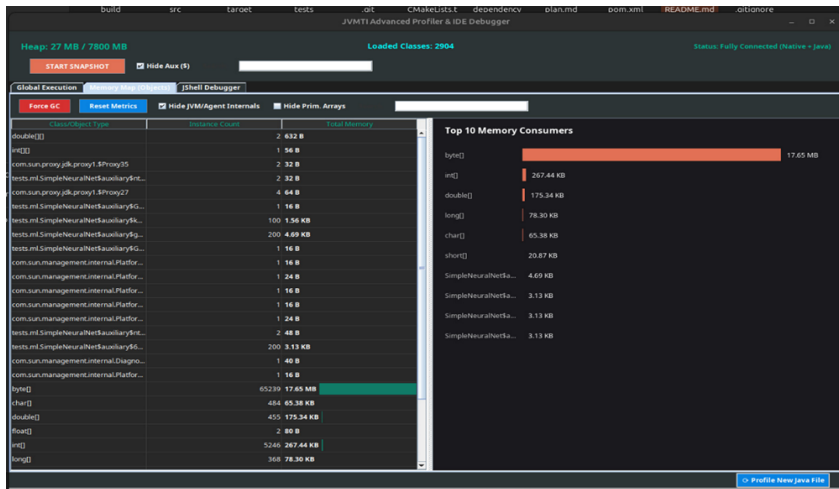


Figure: Displays heap usage per class. **Benefit:** Helps detect memory leaks and optimize usage.

Results: Performance Comparison

Impact of Using the Profiler

Testing shows **consistent improvements** in developer insight, debugging speed, and resource visibility.

Metric	Without Profiler	With Profiler	Improvement
Bottleneck Detection	Manual guesswork	Automated, precise	Instant accuracy
Memory Leak Identification	Hours of log analysis	Minutes via heap chart	~10× faster
Call Flow Understanding	Not available	Visual call graph	Full clarity
Debugging Complexity	High effort	Low (JShell)	Significantly reduced
Thread Activity Tracking	Invisible	Fully tracked	100% visibility
Method-level Timing	Not measurable	nanosecond precision	Granular insight

Table: Profiler benefit comparison for Java applications.

Discussion: Real-World Applications

Performance Optimization

Identifies slow methods and optimizes code for **faster, more efficient** Java applications across all domains.

Enterprise App Monitoring

Monitors large-scale Java systems to ensure **smooth operation** under heavy workloads and high concurrency.

Web Application Tuning

Improves **response times** and user experience by pinpointing slow components in Spring or Jakarta EE apps.

Discussion: Real-World Applications

Performance Optimization

Identifies slow methods and optimizes code for **faster, more efficient** Java applications across all domains.

Education & Learning

Powerful **teaching tool** for students to understand JVM internals, bytecode, and performance analysis hands-on.

Enterprise App Monitoring

Monitors large-scale Java systems to ensure **smooth operation** under heavy workloads and high concurrency.

Testing & Quality Assurance

Evaluates application behaviour under different **workloads during QA** to catch regressions before release.

Web Application Tuning

Improves **response times** and user experience by pinpointing slow components in Spring or Jakarta EE apps.

Resource Usage Analysis

Analyzes CPU usage and execution time for **better resource management** and cloud cost reduction.

Challenges Faced

- **Profiling Overhead** — instrumentation adds extra instructions, slightly slowing the target application.
- **Thread Safety** — concurrent method calls can corrupt shared data structures if not handled carefully.
- **JVM Compatibility** — JVMTI hooks differ across Java versions, breaking profiler on updates.
- **Large Call Graphs** — deeply recursive or highly connected programs produce graphs that are hard to visualize.
- **Data Volume** — high-frequency apps generate massive event streams that must be processed without lag.

Solutions Applied

- **ConcurrentHashMap** — thread-safe storage with no global locks.
- **Selective Instrumentation** — profile only critical packages, not third-party libraries.
- **Async Socket Transfer** — decouples data collection from UI rendering to prevent blocking.
- **Graph Pruning** — collapse small leaf nodes in call graph for readability.
- **Buffered Transmission** — batch events before sending to UI to reduce socket overhead.

Discussion: Optimizations and Future Enhancements

Current Optimizations

- **ConcurrentHashMap** for zero-lock concurrent updates.
- **Buffered socket** transmission reduces UI lag.
- **Selective instrumentation** reduces overhead on non-critical code.
- **Graph pruning** keeps call graph readable even for large apps.
- **Nanosecond timestamps** ensure high-precision timing.

Future Enhancements

- ① **GC Tracking** — monitor garbage collection cycles and pauses.
- ② **Deadlock Detection** — detect and visualize thread deadlocks.
- ③ **Real-time Dashboard** — charts with zoom, filter, and export.
- ④ **Large-Scale Support** — handle high-load enterprise apps.
- ⑤ **Sampling Mode** — statistical profiling for production use.
- ⑥ **Automated Reports** — generate PDF/HTML performance summaries.
- ⑦ **IDE Plugin** — integrate directly into IntelliJ or Eclipse.

Conclusion

- ① **Functionality Verified** — profiler correctly captures execution details across all tested programs.
- ② **Concepts Mastered** — deep understanding of Java agents, bytecode modification, and JVM internals.
- ③ **Practical Skills** — learned to detect slow methods and analyze real-world execution data hands-on.
- ④ **Seamless Integration** — Java agent, JVMTI, socket communication, and Swing UI work as one cohesive system.
- ⑤ **Real-Time Monitoring** — continuous tracking makes debugging and optimization faster and more reliable.
- ⑥ **Educational Value** — serves as a clear, hands-on guide to JVM profiling for students and developers.

Final Thought

“Profiling is not just a tool — it is the lens through which developers see the true behaviour of software and transform complexity into clarity, performance, and reliability.”

-  Burchell, T., & Marr, S. (2023). *Don't Trust Your Profiler: An Empirical Study on the Precision and Accuracy of Java Profilers*. MPLR. [\[PDF Link\]](#)
-  Burchell, T., & Marr, S. (2024). *Towards Realistic Results for Instrumentation-Based Profilers for JIT-Compiled Systems*. MPLR. [\[PDF Link\]](#)
-  Automated Software Engineering Research Team. (2024). *Comparing CPU and Memory Profiler Results Across Multiple Java Versions*. Springer. [\[Link\]](#)
-  Programming Journal Research Team. (2023). *Profiling and Optimizing Java Streams*. Programming Journal. [\[Link\]](#)
-  SoftwareX Research Team. (2024). *JVM Optimization: An Empirical Analysis of JVM Configurations for Enhanced Web Application Performance*. SoftwareX / ScienceDirect. [\[Link\]](#)
-  ACM Research Publication. (2022). *Optimal Heap Limits for Reducing Browser Memory Use*. ACM Digital Library. [\[Link\]](#)

Thank You!

Questions & Feedback Welcome



Dept. of CSE — Aurora Higher Education and Research Academy, Hyderabad

Guide: Dr. Vemula Aruna, Associate Professor